

Лабораторная работа №2

Структуры данных

Александр Эдуардович Аскеров

2025-09-27

Докладчик

- Аскеров Александр Эдуардович
- Кафедра теории вероятностей и кибербезопасности
- Российский университет дружбы народов им. П. Лумумбы

Цель работы

Основная цель работы – изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

Задание

1. Используя Jupyter Lab, повторите примеры из раздела 2.2.
2. Выполните задания для самостоятельной работы (раздел 2.4).

Выполним примеры из раздела кортежи.

Кортеж (Tuple) – структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы).

```
In [19]: {}  
Out[19]: {}  
  
In [20]: favoritelang = ("Python", "Julia", "R")  
Out[20]: ("Python", "Julia", "R")  
  
In [21]: x1 = (1, 2, 3)  
Out[21]: (1, 2, 3)  
  
In [22]: x2 = (1, 2.0, "tmp")  
Out[22]: (1, 2.0, "tmp")  
  
In [23]: x3 = (a=2, b=1+2)  
Out[23]: (a = 2, b = 3)
```

Рисунок 1: Кортежи 1

```
In [24]: length(x2)
Out[24]: 3

In [25]: x2[1], x2[2], x2[3]
Out[25]: (1, 2.0, "tmp")

In [26]: c = x1[2] + x1[3]
Out[26]: 5

In [27]: x3.a, x3.b, x3[2]
Out[27]: (2, 3, 3)

In [28]: in("tmp", x2), 0 in x2
Out[28]: (True, False)
```

Рисунок 2: Кортежи 2

Выполним примеры из раздела словари.

Словарь – неупорядоченный набор связанных между собой по ключу данных.

```
In [29]: phonebook = Dict{  
          "Иванов И.И." => ("867-5309", "333-5544"),  
          "Бухгалтерия" => "555-2368"  
        }  
  
Out[29]: Dict{String, Any} with 2 entries:  
         "Бухгалтерия" => "555-2368"  
         "Иванов И.И." => ("867-5309", "333-5544")  
  
In [30]: keys(phonebook)  
  
Out[30]: KeySet for a Dict{String, Any} with 2 entries. Keys:  
         "Бухгалтерия"  
         "Иванов И.И."  
  
In [31]: values(phonebook)  
  
Out[31]: ValueIterator for a Dict{String, Any} with 2 entries. Values:  
         "555-2368"  
         ("867-5309", "333-5544")  
  
In [32]: pairs(phonebook)  
  
Out[32]: Dict{String, Any} with 2 entries:  
         "Бухгалтерия" => "555-2368"  
         "Иванов И.И." => ("867-5309", "333-5544")
```

Рисунок 3: Словари 1

```
In [33]: haskey(phonebook, "Иванов И.И.")
Out[33]: true

In [34]: phonebook["Орлов П.С."] = "555-3344"
Out[34]: "555-3344"

In [35]: pop!(phonebook, "Иванов И.И.")
Out[35]: ("867-5309", "333-5544")

In [36]: a = Dict{String, Real}{"foo" => 0.0, "bar" => 42.0};
          b = Dict{String, Real}{"bar" => 17, "baz" => 13.0};
          merge(a, b)
Out[36]: Dict{String, Real}{"bar" => 13.0, "baz" => 17, "foo" => 0.0}, Dict{String, Real}{"bar" => 42.0, "baz" => 17, "foo" => 0.0}
```

Рисунок 4: Словари 2

Выполним примеры из раздела множества.

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа. Возможные операции над множествами: объединение, пересечение, разность; принадлежность элемента множеству.

```
In [27]: A = Set{1, 3, 4, 5}
Out[27]: Set{Int64} with 4 elements:
  5
  4
  3
  1

In [38]: B = Set{"abracadabra"}
Out[38]: Set{Char} with 5 elements:
  'a'
  'b'
  'r'
  'c'
  'd'

In [39]: S1 = Set{1, 2}
          S2 = Set{3, 4}
          issetequal(S1, S2)
Out[39]: false

In [40]: S3 = Set{1, 2, 2, 3, 1, 2, 3, 2, 1}
          S4 = Set{2, 3, 1}
          issetequal(S3, S4)
Out[40]: true
```

Рисунок 5: Множества 1


```
Out[40]: true

In [43]: C = union(S1, S2)
Out[43]: Set{Int64} with 4 elements:
         4
         2
         3
         1

In [44]: D = intersect(S1, S3)
Out[44]: Set{Int64} with 2 elements:
         2
         1

In [45]: E = setdiff(S3, S1)
Out[45]: Set{Int64} with 1 element:
         3

In [46]: issubset(S1, S4)
Out[46]: true

In [47]: push!(S4, 99)
Out[47]: Set{Int64} with 4 elements:
         2
        99
         3
         1

In [48]: pop!(S4)
Out[48]: 2
```

Рисунок 6: Множества 2

Выполним примеры из раздела массивы.

Массив – коллекция упорядоченных элементов, размещённая в многомерной сетке.

Векторы и матрицы являются частными случаями массивов.

```
In [49]: empty_array = []
Out[49]: Any[]

In [52]: empty_array_2 = (Int64){}
Out[52]: Int64[]

In [53]: empty_array_3 = (Float64){}
Out[53]: Float64[]

In [54]: a = [1, 2, 3]
Out[54]: 3-element Vector{Int64}:
 1
 2
 3

In [55]: b = [1 2 3]
Out[55]: 1×3 Matrix{Int64}:
 1 2 3

In [56]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
Out[56]: 3×3 Matrix{Int64}:
 1 4 7
 2 5 8
 3 6 9
```

Рисунок 7: Массивы 1

```
In [57]: B = [[1 2 3]; [4 5 6]; [7 8 9]]
Out[57]: 3x3 Matrix{Int64}:
  1  2  3
  4  5  6
  7  8  9

In [58]: C = rand(1, 8)
Out[58]: 1x8 Matrix{Float64}:
 0.16822  0.278178  0.0879462  0.485416  - 0.777814  0.829817  0.89761
1

In [59]: C = rand(2, 3)
Out[59]: 2x3 Matrix{Float64}:
 0.187428  0.538214  0.875849
 0.851585  0.472582  0.795135
```

Рисунок 8: Массивы 2

```
Out[59]: 2x3 Matrix{Float64}:  
 0.187428  0.538214  0.875849  
 0.851585  0.472582  0.795135
```

```
In [62]: D = rand{4, 3, 2}
```

```
Out[62]: 4x3x2 Array{Float64, 3}:  
[:, :, 1] =  
 0.272256  0.638178  0.10435  
 0.495959  0.83931  0.845109  
 0.0489139 0.141009  0.416999  
 0.315541  0.636198  0.140045  
  
[:, :, 2] =  
 0.576745  0.00782045  0.179376  
 0.854092  0.799871  0.257626  
 0.46979  0.694563  0.537992  
 0.465904  0.674004  0.334386
```

```
In [63]: roots = [sqrt(i) for i in 1:10]
```

```
Out[63]: 10-element Vector{Float64}:  
 1.0  
 1.4142135623730951  
 1.7320508075688772  
 2.0  
 2.23606797749979  
 2.449489742783178  
 2.6457513110645907  
 2.8284271247461903  
 3.0  
 3.1622776601683795
```

Рисунок 9: Массивы 3

```
In [63]: roots = [sqrt(i) for i in 1:10]
Out[63]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795

In [64]: ar_1 = [3*i^2 for i in 1:2:9]
Out[64]: 5-element Vector{Int64}:
 3
 27
 75
 147
 243

In [65]: ar_2 = [i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
Out[65]: 4-element Vector{Int64}:
 1
 9
 49
 81
```

Рисунок 10: Массивы 4

```
In [56]: ones(5)
Out[56]: 5-element Vector{Float64}:
 1.0
 1.0
 1.0
 1.0
 1.0

In [57]: ones(2, 3)
Out[57]: 2x3 Matrix{Float64}:
 1.0  1.0  1.0
 1.0  1.0  1.0

In [58]: zeros(4)
Out[58]: 4-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0

In [59]: fill(3.5, (3, 2))
Out[59]: 3x2 Matrix{Float64}:
 3.5  3.5
 3.5  3.5
 3.5  3.5
```

Рисунок 11: Массивы 5

```
In [70]: repeat([1, 2], 3, 3)
```

```
Out[70]: 6x3 Matrix{Int64}:  
 1 1 1  
 2 2 2  
 1 1 1  
 2 2 2  
 1 1 1  
 2 2 2
```

```
In [71]: repeat([1 2], 3, 3)
```

```
Out[71]: 3x6 Matrix{Int64}:  
 1 2 1 2 1 2  
 1 2 1 2 1 2  
 1 2 1 2 1 2
```

```
In [72]: a = collect(1:12)
```

```
Out[72]: 12-element Vector{Int64}:  
 1  
 2  
 3  
 4  
 5  
 6  
 7  
 8  
 9  
10  
11  
12
```

Рисунок 12: Массивы 6

```
In [73]: b = reshape(a, (2, 6))  
Out[73]: 2x6 Matrix{Int64}:  
 1 3 5 7 9 11  
 2 4 6 8 10 12  
  
In [74]: b'  
Out[74]: 6x2 adjoint{::Matrix{Int64}} with eltype Int64:  
 1 2  
 3 4  
 5 6  
 7 8  
 9 10  
 11 12  
  
In [75]: c = transpose(b)  
Out[75]: 6x2 transpose{::Matrix{Int64}} with eltype Int64:  
 1 2  
 3 4  
 5 6  
 7 8  
 9 10  
 11 12
```

Рисунок 13: Массивы 7


```
In [76]: ar = rand(10:20, 10, 5)
```

```
Out[76]: 10x5 Matrix{Int64}:  
 17 15 17 12 10  
 16 10 20 14 12  
 16 13 19 16 10  
 16 18 10 10 10  
 16 16 15 17 15  
 17 10 11 10 18  
 19 13 10 14 11  
 16 13 20 16 10  
 14 13 10 18 18  
 11 16 15 19 17
```

```
In [77]: ar[:, 2]
```

```
Out[77]: 10-element Vector{Int64}:  
 15  
 10  
 13  
 18  
 16  
 10  
 13  
 13  
 13  
 16
```

Рисунок 14: Массивы 8

```
In [78]: ar[:, [2, 5]]  
Out[78]: 10x2 Matrix{Int64}:  
15 10  
10 12  
13 10  
18 10  
16 15  
10 18  
13 11  
13 10  
13 18  
16 17  
  
In [80]: ar[:, 2:4]  
Out[80]: 10x3 Matrix{Int64}:  
15 17 12  
10 20 14  
13 19 16  
18 10 10  
16 15 17  
10 11 10  
13 10 14  
13 20 16  
13 10 18  
16 15 19  
  
In [81]: ar[[2, 4, 6], [1, 5]]  
Out[81]: 3x2 Matrix{Int64}:  
16 12  
16 10  
17 18
```

Рисунок 15: Массивы 9

```
In [82]: ar[1, 3:end]
```

```
Out[82]: 3-element Vector{Int64}:  
 17  
 12  
 10
```

```
In [83]: sort(ar, dims=1)
```

```
Out[83]: 10x5 Matrix{Int64}:  
 11 10 10 10 10  
 14 10 10 10 10  
 16 13 10 12 10  
 16 13 11 14 10  
 16 13 15 14 11  
 16 13 15 16 12  
 16 15 17 16 15  
 17 16 19 17 17  
 17 16 20 18 18  
 19 18 20 19 18
```

```
In [84]: sort(ar, dims=2)
```

```
Out[84]: 10x5 Matrix{Int64}:  
 10 12 15 17 17  
 10 12 14 16 20  
 10 13 16 16 19  
 10 10 10 16 18  
 15 15 16 16 17  
 10 10 11 17 18  
 10 11 13 14 19  
 10 13 16 16 20  
 10 13 14 18 18  
 11 15 16 17 19
```

Рисунок 16: Массивы 10

```
In [85]: ar .> 14
Out[85]: 10x5 BitMatrix:
  1 1 1 0 0
  1 0 1 0 0
  1 0 1 1 0
  1 1 0 0 0
  1 1 1 1 1
  1 0 0 0 1
  1 0 0 0 0
  1 0 1 1 0
  0 0 0 1 1
  0 1 1 1 1
```

Рисунок 17: Массивы 11

```
In [86]: findall(ar .> 14)
Out[86]: 27-element Vector{CartesianIndex{2}}:
 CartesianIndex{1, 1}
 CartesianIndex{2, 1}
 CartesianIndex{3, 1}
 CartesianIndex{4, 1}
 CartesianIndex{5, 1}
 CartesianIndex{6, 1}
 CartesianIndex{7, 1}
 CartesianIndex{8, 1}
 CartesianIndex{1, 2}
 CartesianIndex{4, 2}
 CartesianIndex{5, 2}
 CartesianIndex{10, 2}
 CartesianIndex{1, 3}
      :
 CartesianIndex{5, 3}
 CartesianIndex{8, 3}
 CartesianIndex{10, 3}
 CartesianIndex{3, 4}
 CartesianIndex{5, 4}
 CartesianIndex{8, 4}
 CartesianIndex{9, 4}
 CartesianIndex{10, 4}
 CartesianIndex{5, 5}
 CartesianIndex{6, 5}
 CartesianIndex{9, 5}
 CartesianIndex{10, 5}
```

Рисунок 18: Массивы 12

Задание 1. Работа со множествами.

```
In [88]: A = Set([0, 3, 4, 9])
         B = Set([1, 3, 4, 7])
         C = Set([0, 1, 2, 4, 7, 8, 9])
         P = intersect(union(A, B), union(A, B), union(A, C), union(B, C))

Out[88]: Set{Int64} with 6 elements:
         0
         4
         7
         9
         3
         1
```

Рисунок 19: Работа со множествами

Задания для самостоятельной работы

Задание 2. Операции над множествами элементов разных типов.

```
In [99]: aaa = (12, 13, 14); bbb = [444, 4421, 3456]
length(aaa)

Out[99]: 3

In [100]: ccc = [aaa[end], pop!(bbb), "RRR"]

Out[100]: 3-element Vector{Any}:
 14
 3456
 "RRR"

In [102]: ddd = [i for i in ccc]

Out[102]: 3-element Vector{Any}:
 14
 3456
 "RRR"
```

Рисунок 20: Операции над множествами элементов разных типов

Задания для самостоятельной работы

Задание 3. Создание разными способами массивов и векторов.

Задание 3.1.

```
ar = [i for i in 1:21]
print(ar)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
```

Рисунок 21: Задание 3.1

Задание 3.2.

```
ar = reverse([i for i in 1:21])
print(ar)

[21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

ar = [i for i in 21:-1:1]
print(ar)

[21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Рисунок 22: Задание 3.2

Задания для самостоятельной работы

Задание 3.3.

```
ar = vcat(1:21, 20:-1:1)
print(ar)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2,
1]
```

Рисунок 23: Задание 3.3

Задание 3.4.

```
tmp = [4 6 3]
print(tmp)

[4 6 3]
```

Рисунок 24: Задание 3.4

Задания для самостоятельной работы

Задание 3.5.

```
ar = vcat([tmp[1] for _ in 1:10], tmp[2:end])  
print(ar)  
  
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 3]
```

Рисунок 25: Задание 3.5

Задание 3.6.

```
ar = vcat([tmp[1] for _ in 1:10], [tmp[2] for _ in 1:10],  
          [tmp[3] for _ in 1:10])  
print(ar)  
  
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3,  
 3, 3, 3, 3, 3]
```

Рисунок 26: Задание 3.6

Задания для самостоятельной работы

Задание 3.7.

```
ar = vcat([tmp[1] for _ in 1:11], [tmp[2] for _ in 1:10],  
          [tmp[3] for _ in 1:10])  
print(ar)
```

[4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3,
 3, 3, 3, 3, 3, 3]

Рисунок 27: Задание 3.7

Задание 3.8.

```
ar = vcat([tmp[1] for _ in 1:10], [tmp[2] for _ in 1:20],  
          [tmp[3] for _ in 1:30])  
print(ar)
```

Рисунок 28: Задание 3.8

Задания для самостоятельной работы

Задание 3.9.

```
tmp = [4 6 3]
ar = []
for i in tmp
    push!(ar, 2^i)
end
ar = vcat(ar[1:2], [ar[3] for _ in 1:4])
ar = string.(ar)
ar = join(ar, "")
counter = count(isequal('6'), ar)
print(counter)
```

2

Рисунок 29: Задание 3.9

Задание 3.10.

```
y(x) = exp(x)*cos(x)
Y = [y(x) for x in 3:0.1:6]
mean = sum(Y) / length(Y)
```

53.11374594642971

Рисунок 30: Задание 3.10

Задания для самостоятельной работы

Задание 3.11.

```
vec = {}
for j in 1:3:34
    i = j + 2
    push!(vec, {0.1*i, 0.2*j})
end
print(vec)
```

```
Any{()}.0e10000000000000000e2, 0.2), {1.0e000000000000000e-6, 0.0016000000000000000e3}, {1.0000000000000000e-9, 1.2800000000000000e-5}, {1.0000000000000000e-12, 0.02400000000000000e-7}, {1.0000000000000000e-15, 0.19200000000000000e-10}, {1.0000000000000000e-18, 0.6536000000000000e-12}, {1.0000000000000000e-21, 5.242880000000000e-14}, {1.0000000000000000e-24, 1.943600000000000e-16}, {1.0000000000000000e-27, 3.955440000000000e-18}, {1.0000000000000000e-30, 2.664448000000000e-20}, {1.0000000000000000e-33, 2.147483360000000e-22}, {1.0000000000000000e-36, 1.717969184000000e-24}
```

Рисунок 31: Задание 3.11

Задание 3.12.

[illegible]

Рисунок 32: Задание 3.12

Задания для самостоятельной работы

Задание 3.13.

```
vect = []  
for i in 1:30  
    push!(vect, "fn$i")  
end  
print(vect)
```

Any["fn1", "fn2", "fn3", "fn4", "fn5", "fn6", "fn7", "fn8", "fn9", "fn10", "fn11", "fn12", "fn13", "fn14", "fn15", "fn16", "fn17", "fn18", "fn19", "fn20", "fn21", "fn22", "fn23", "fn24", "fn25", "fn26", "fn27", "fn28", "fn29", "fn30"]

Рисунок 33: Задание 3.13

Задания для самостоятельной работы

Задание 3.14.

```
vect = []
for i in 1:249
    j = i + 1
    push!(vect, y[j] - x[i])
end
print(vect)
```

Any[281, 59, 188, -470, 587, -699, 370, 126, -434, -96, -374, 771, 449,
-126, 892, 98, -568, 221, -129, -687, -313, 786, -8, 192, -333, -190,
-389, 350, -562, 263, -265, -948, -27, 200, 82, -351, -299, -173, 232,
-481, -506, -642, -47, -459, -583, 132, 379, -228, 204, -217, 512, -26
9, -372, 169, -287, -216, 357, -279, -257, -117, -21, -189, 480, -498,
697, 295, 23, 693, 115, -44, 362, -259, 179, -407, 66, 175, 7, 250, 398
, -708, 82, -145, -190, 548, -37, -202, -43, -164, 87, 571, 300, -424,
-267, -368, -599, -749, 91, -809, 50, 134, -55, 66, 668, 317, 547, 572,
231, 162, -445, 180, 191, 182, -354, 110, -150, 825, -46, 569, -576, 51
5, -128, -234, -659, -69, -455, 277, -189, 738, 84, 515, 178, -508, 37
4, 248, -237, 430, 373, -180, 38, 58, 785, -248, -143, 127, -74, -575,
673, -401, -554, 47, -334, -389, -615, 146, 23, -517, -476, -767, 63,
-853, -140, 377, 144, -616, -399, 591, 400, -544, 521, 654, 823, -250,
427, 485, 254, 158, -169, 783, -94, 320, -402, 450, 118, -282, 25, -45,
-871, -913, -288, -563, -646, 798, 116, -38, 205, 230, 464, -252, -274,
-929, 657, -350, 98, -360, 66, -300, 123, -88, -59, 83, 0, 424, 480, -9
3, 22, 35, -565, -337, -279, 347, 592, -489, -96, -619, -391, -212, -39
4, -488, 655, 517, -106, -3, 421, -741, -3, -258, 415, 67, 38, 365, 748
, 288, -873, -351, 329, 906, 537, -535, -125]

Рисунок 34: Задание 3.14.1

Задания для самостоятельной работы

```
vect = [x[i] + 2*x[i+1] - x[i+2] for i in 1:248]
print(vect)
```

[1824, 79, 1735, 641, 1855, 1690, 44, 1030, 1297, 2210, 687, 663, 1235, 196, -23, 968, 1404, 179, 1776, 1952, 253, 342, 1001, 270, 1236, 2609, 190, 1121, 1697, 607, 2190, 1104, 394, 429, 610, 1093, 1046, 388, 1043, 1676, 2621, 655, 677, 2094, 1610, 235, 1862, 1534, 910, 17, 708, 1085, 523, 1752, 1556, -432, 1237, 1055, 1150, 1005, 464, -364, 1555, 791, -5, 83, 1183, 442, 979, 1571, 578, 899, 1471, 1253, 2029, 898, 1684, 1190, -114, 1192, 722, 1232, 2577, 760, 874, 1443, 761, 1011, 75, 454, 506, 1, 181, 1124, 1598, 1920, 2385, 496, 1758, 700, 660, 1006, 407, -127, 344, 764, 326, 1263, 796, 1619, 531, 1614, 376, 332, 1337, 2134, 131, 1560, 415, 1700, 1238, 1081, 1564, 2237, 640, 795, 1473, 1773, -109, 1395, 60, 5, 428, 816, 1845, 98, 1667, 378, 338, 1126, 1426, 926, 194, 801, 1526, 1547, 1299, 1691, 569, 201, 1700, 1367, 1245, 1659, 1692, 1071, -51, 14, 55, 1378, 1697, 1380, 2168, 1541, 780, 825, 1994, 1614, 1154, -219, 830, -1455, 258, -48, 503, 790, 796, 126, -279, 1664, 600, 761, 195, 1372, -250, 991, 1545, 1660, 1345, 1649, 2295, 1112, 1066, 2354, 837, 259, 33, 7, 683, 543, 852, 290, 747, 2412, 528, 1340, 1117, 624, 805, 1495, 1455, -184, 1921, 295, 934, 1142, -60, 839, 1559, 50, 877, 1184, 1050, 911, -706, 1412, 461, 1295, 2093, 1201, 440, 2382, 771, 213, 352, 315, 397, 1857, 979, 1660, 726, 606, 1893, 1171, 353, -425, 1569, 2003, 1077, 362, -896, 1760, 647]

Рисунок 35: Задание 3.14.2

Задания для самостоятельной работы

```
vect = [round(sin(y[i])/cos(x[i+1]), digits=2) for i in 1:249]
print(vect)

[-0.07, 0.92, 1.06, -1.49, 0.84, 1.91, -0.16, -0.38, 0.91, 0.86, 1.69,
0.80, 0.49, 0.89, 0.42, 0.36, 4.1, 0.5, -0.77, 0.94, -1.53, 1.36, 1.0
-0.83, -0.09, -0.52, -1.12, -1.07, 1.29, -0.14, -0.8, 0.87, -3.14,
-1.46, -6.19, -0.57, -0.94, -0.28, 0.36, 1.22, -1.82, 0.62, -1.54, -0.8
6, 2.7, 3.49, 3.0, 4.84, 0.58, 0.31, 0.18, 0.76, 1.06, 0.99, 1.16,
-1.16, 1.33, 3.29, -2.79, 0.83, -1.28, 1.34, 1.32, -0.9, 5.09, 0.64,
-1.61, 0.87, 1.17, -6.42, -1.65, 0.69, -0.48, 0.32, -0.68, -1.38, 3.48,
-4.58, 0.72, -3.29, -1.64, 0.98, -9.96, 1.08, 0.55, 0.9, 1.01, -2.42,
0.54, -0.95, 0.9, 0.86, 0.31, 0.31, 0.11, 0.57, -1.12, 0.17, -0.94,
-5.82, -1.25, -0.12, 0.61, -222.03, 0.22, -1.1, 0.83, 1.23, 1.08, 205.0
9, 2.52, -3.91, 0.96, -0.71, -0.5, 0.07, -3.15, 0.5, -0.83, 1.04, -0.0
3, -1.89, 1.65, -1.12, -0.89, -1.22, -2.01, -1.73, 0.77, -0.79, 0.33, 1
.05, -1.18, 1.85, -0.77, -0.35, -0.62, 11.58, -1.19, 1.63, -0.82, 2.52,
-2.03, 0.57, -0.05, 0.98, 0.67, -0.22, -1.12, 213.76, -3.3, 4.99, -0.5
9, 1.04, 1.63, -0.11, 0.43, 1.01, 0.38, -0.66, 0.74, -2.91, -1.33, -4.6
9, 1.14, -0.99, 1.03, -5.13, -0.63, -1.03, -0.84, -0.89, 1.55, 0.67, 1.
06, -0.44, -1.25, 0.94, -1.39, -1.39, -1.44, -1.44, 3.75, 0.88, 0.39, 1
3.77, -0.7, -1.78, 0.96, -0.52, -0.7, -1.0, 0.69, -0.51, -0.64, -1.27,
3.91, 2.4, 9.83, -0.53, 0.7, -1.12, 3.06, 0.53, -0.02, -0.78, -1.01, 3.
28, 0.41, 1.06, 1.11, 1.44, -1.46, -0.27, 1.17, -1.63, -1.11, 1.79,
-1.17, -6.87, -0.8, 0.04, 2.94, 1.01, -1.05, -1.23, 2.48, -1.09, 0.89,
1.36, -1.25, -0.1, -2.11, -0.34, -0.71, -0.44, -0.42, 0.77, -1.6, -0.6
6, 0.35, 0.75, -1.11, -0.56, -0.58, 1.02, -0.41, -0.09, -2.1]
```

Рисунок 36: Задание 3.14.3

```
sum([exp(-x[i+1]) / (x[i] + 10) for i in 1:249])
3.261995204084063e-5
```

Рисунок 37: Задание 3.14.4

Задания для самостоятельной работы

```
for i in 1:length(y)
    if y[i] > 600
        print(i, " ", y[i], " ");
    end
end
```

2) 651; 2) 850; 6) 916; 8) 814; 13) 924; 14) 987; 16) 927; 19) 769; 23) 847; 25) 673; 27) 785; 31) 868; 40) 685; 47) 639; 48) 729; 49) 744; 50) 726; 52) 638; 55) 605; 60) 738; 68) 708; 69) 912; 70) 797; 72) 681; 74) 927; 76) 937; 78) 784; 83) 731; 84) 765; 85) 757; 91) 978; 92) 799; 95) 619; 104) 718; 105) 647; 106) 913; 107) 890; 108) 907; 112) 912; 115) 898; 117) 951; 118) 811; 119) 841; 121) 959; 123) 601; 127) 978; 129) 758; 130) 789; 131) 650; 138) 837; 140) 602; 142) 973; 144) 638; 145) 820; 148) 878; 151) 634; 160) 798; 163) 835; 164) 793; 167) 942; 170) 936; 171) 689; 172) 970; 174) 745; 175) 668; 178) 620; 179) 850; 184) 986; 186) 846; 187) 678; 193) 898; 198) 878; 202) 835; 206) 626; 208) 626; 210) 880; 213) 753; 230) 667; 231) 727; 234) 917; 238) 684; 239) 633; 240) 833; 241) 628; 242) 898; 247) 968;

Рисунок 38: Задание 3.14.5

```
for i in 1:length(y)
    if y[i] > 600
        print(i, " ", x[i], " ");
    end
end
```

2) 791; 3) 128; 6) 985; 8) 183; 13) 458; 14) 406; 16) 280; 19) 310; 23) 381; 25) 342; 27) 896; 31) 487; 40) 794; 47) 350; 48) 972; 49) 432; 50) 302; 52) 537; 55) 841; 60) 30; 68) 219; 69) 682; 70) 604; 72) 664; 74) 609; 76) 402; 78) 272; 83) 955; 84) 209; 85) 613; 91) 409; 92) 719; 95) 922; 104) 330; 105) 366; 106) 318; 107) 676; 108) 405; 112) 86; 115) 73; 117) 857; 118) 272; 119) 986; 121) 716; 123) 822; 127) 546; 129) 755; 130) 135; 131) 428; 138) 613; 140) 315; 142) 657; 144) 693; 145) 620; 148) 459; 151) 721; 160) 973; 163) 649; 164) 931; 167) 25; 170) 35; 171) 147; 172) 877; 174) 263; 175) 48; 178) 147; 179) 393; 184) 749; 186) 723; 187) 922; 193) 284; 198) 272; 202) 769; 206) 719; 208) 278; 210) 67; 213) 114; 230) 210; 231) 219; 234) 891; 238) 566; 239) 795; 240) 283; 241) 150; 242) 210; 247) 16;

Рисунок 39: Задание 3.14.6

Задания для самостоятельной работы

```
using Statistics
vect = [round(abs(i - mean(x))^(1/2), digits=2) for i in x]
print(vect)

[11.4, 17.06, 19.29, 21.63, 13.08, 22.02, 7.48, 17.8, 16.31, 13.6, 18.4
1, 18.63, 6.48, 9.69, 21.36, 14.83, 10.86, 6.93, 13.78, 22.11, 3.47, 20
.95, 10.91, 4.35, 12.57, 19.88, 19.9, 20.54, 18.82, 12.85, 3.6, 21.26,
17.29, 15.81, 13.89, 8.66, 6.86, 8.0, 11.27, 17.15, 20.45, 21.31, 17.17
, 14.46, 21.12, 2.65, 12.25, 21.71, 8.24, 14.07, 19.34, 6.89, 2.82, 8.0
, 18.47, 11.62, 22.02, 18.88, 10.86, 6.0, 14.0, 19.0, 19.54, 15.46, 21
.42, 21.68, 13.64, 16.76, 13.49, 10.2, 13.45, 12.81, 15.75, 13.75, 19.26
, 9.9, 16.64, 15.1, 18.14, 15.1, 8.86, 19.39, 21.33, 17.86, 10.63, 7.81
, 14.42, 10.77, 21.24, 9.64, 9.54, 14.8, 12.89, 20.66, 20.54, 18.74, 16
.15, 18.25, 18.79, 8.54, 12.61, 19.92, 21.21, 13.04, 11.57, 13.49, 13.2
7, 9.74, 13.36, 15.36, 14.87, 20.35, 4.13, 16.97, 15.36, 19.34, 18.9, 1
5.1, 22.05, 4.0, 14.7, 18.3, 17.95, 16.67, 10.82, 14.18, 6.78, 21.91, 1
5.97, 19.1, 8.94, 6.86, 14.07, 20.83, 17.21, 21.84, 6.0, 10.63, 8.0, 13
.6, 13.23, 12.53, 16.76, 13.89, 10.96, 11.58, 17.41, 6.4, 20.35, 9.33,
14.87, 16.85, 11.4, 12.16, 15.39, 20.71, 12.89, 19.57, 15.33, 21.75, 3.
61, 6.48, 12.21, 20.76, 4.13, 12.21, 21.79, 10.96, 8.86, 21.36, 18.79,
11.09, 13.49, 13.39, 21.26, 16.34, 17.0, 16.79, 10.34, 18.11, 10.1, 22
.27, 19.18, 15.78, 17.92, 14.93, 20.54, 20.45, 6.08, 15.23, 19.0, 20.0,
17.2, 15.84, 11.62, 14.28, 9.27, 15.1, 14.42, 21.0, 17.94, 16.4, 11.13,
9.8, 7.75, 14.8, 1.74, 15.16, 18.95, 20.81, 9.9, 13.08, 19.63, 11.09, 4
.59, 19.85, 13.53, 9.75, 13.75, 15.62, 22.18, 18.36, 15.16, 20.4, 17.52
, 7.93, 4.47, 21.38, 22.07, 17.83, 16.76, 14.28, 1.99, 19.77, 8.89, 15
.94, 15.2, 8.13, 17.18, 15.39, 18.71, 17.83, 22.25, 11.43, 15.68, 20.93,
22.0, 22.14, 16.25, 17.75]
```

Рисунок 40: Задание 3.14.7

```
length([i for i in y if abs(maximum(y) - i) <= 200])
```

42

Рисунок 41: Задание 3.14.8

Задания для самостоятельной работы

```
println("Even: ", count{true for i in x if i%2==0})  
println("Odd: ", count{true for i in x if i%2!=0})  
Even: 123  
Odd: 127
```

Рисунок 42: Задание 3.14.9

```
println("Devides by 7: ", count{true for i in x if i%7==0})  
Devides by 7: 44
```

Рисунок 43: Задание 3.14.10

Задания для самостоятельной работы

```
i = sortperm(y)
sorted_x = x[i]
print(sorted_x)
```

[201, 463, 895, 178, 352, 435, 708, 147, 118, 698, 560, 548, 918, 914, 197, 435, 547, 126, 957, 239, 815, 296, 735, 351, 595, 430, 513, 631, 788, 536, 318, 407, 683, 15, 508, 421, 617, 709, 701, 222, 50, 861, 250, 916, 989, 859, 883, 427, 61, 4, 365, 160, 49, 952, 486, 41, 436, 384, 946, 732, 689, 373, 452, 517, 35, 354, 871, 139, 444, 183, 529, 258, 490, 666, 807, 172, 512, 233, 766, 205, 918, 517, 068, 249, 851, 98, 686, 833, 685, 796, 270, 665, 28, 292, 587, 507, 827, 481, 268, 458, 618, 784, 435, 789, 927, 748, 781, 484, 115, 954, 666, 503, 437, 754, 304, 376, 854, 620, 941, 721, 307, 464, 868, 226, 8, 153, 12, 404, 630, 602, 269, 329, 263, 993, 78, 564, 876, 171, 414, 521, 186, 634, 821, 382, 900, 365, 23, 319, 341, 736, 728, 256, 561, 777, 370, 126, 62, 835, 835, 623, 739, 547, 329, 887, 822, 315, 794, 841, 922, 147, 719, 270, 150, 795, 721, 537, 693, 350, 366, 420, 791, 210, 48, 342, 922, 664, 566, 147, 896, 219, 330, 302, 219, 972, 955, 30, 432, 263, 114, 613, 755, 209, 310, 272, 135, 931, 604, 973, 719, 67, 272, 183, 620, 263, 649, 769, 613, 986, 723, 881, 128, 893, 487, 459, 272, 676, 736, 304, 218, 486, 485, 682, 86, 318, 985, 891, 458, 280, 689, 402, 25, 857, 35, 16, 377, 657, 409, 546, 749, 710]

Рисунок 44: Задание 3.14.11

```
top_10 = partialsort(x, 1:10, rev=true)
print(top_10)
```

[995, 990, 989, 986, 985, 973, 972, 968, 957, 955]

Рисунок 45: Задание 3.14.12

Задания для самостоятельной работы

```
temp_x = Set(x)
temp_x = collect(temp_x)
print(temp_x)
```

```
[482, 35, 689, 719, 38, 723, 941, 67, 219, 929, 319, 272, 871, 657, 153,
115, 547, 420, 404, 861, 139, 665, 365, 4, 485, 86, 263, 183, 376, 63,
12, 676, 23, 382, 41, 171, 487, 895, 857, 666, 486, 517, 126, 955, 1,
72, 685, 382, 103, 598, 288, 521, 833, 837, 387, 750, 366, 414, 108, 33,
2, 484, 28, 721, 754, 323, 135, 512, 222, 732, 489, 481, 918, 972, 851,
249, 381, 839, 916, 289, 957, 236, 258, 859, 649, 78, 568, 998, 436, 25,
8, 564, 735, 8, 435, 789, 281, 427, 684, 587, 686, 464, 278, 777, 628,
968, 548, 682, 868, 716, 488, 432, 789, 634, 254, 496, 566, 683, 595, 3,
15, 269, 114, 794, 827, 268, 985, 197, 748, 444, 784, 233, 342, 954, 21,
0, 239, 736, 698, 393, 946, 351, 285, 739, 821, 62, 682, 437, 158, 766,
896, 587, 952, 851, 989, 791, 854, 623, 98, 284, 118, 377, 883, 537, 91,
4, 25, 318, 384, 292, 756, 973, 50, 318, 728, 296, 384, 788, 769, 147,
927, 458, 16, 378, 341, 887, 49, 664, 693, 788, 749, 583, 186, 61, 841,
876, 513, 995, 631, 128, 358, 835, 815, 463, 781, 561, 922, 421, 373, 4,
92, 487, 546, 613, 178, 617, 536, 459, 822, 931, 986, 795, 781, 618, 48,
13, 338]
```

Рисунок 46: Задание 3.14.13

Задания для самостоятельной работы

Задание 4. Квадраты целых чисел от 1 до 100.

```
squares = [i**2 for i in range(1,101)]  
print(squares)
```

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 1849, 1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356, 4489, 4624, 4761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561, 6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 8836, 9025, 9216, 9409, 9604, 9801, 10000]

Рисунок 47: Квадраты целых чисел от 1 до 100

Задания для самостоятельной работы

Задание 5. Операции с простыми числами.

```
using Primes
myprimes = primes(prime(100))
print(myprimes)

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139,
149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227,
229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307,
311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389,
397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467,
479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577,
587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659,
661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757,
761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857,
859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941, 947, 953, 967,
971, 977, 983, 991, 997]
```

```
prime(89)

461
```

```
print(myprimes[89:99])

[461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]
```

Рисунок 48: Операции с простыми числами

Задания для самостоятельной работы

Задание 6. Вычисления.

```
summa = 0
for i in 10:100
    summa += i^3 + 4*i^2
end
print(summa)
```

26852735

```
N = 25
summa = 0
for i in 1:M
    summa += (2^i)/i + (3^i)/(i^2)
end
print(summa)
```

2.1291704368143802e9

```
N = 38
summa = 1
a_n = 1
for i in 2:2:N
    a_n *= 1/(i+1)
    summa += a_n
end
print(summa)
```

6.976346137897618

Рисунок 49: Вычисления

Изучены несколько структур данных, реализованных в Julia, их применение и операции над ними для решения задач.